IS 6420

Database Theory and Design

EVENTBRITE

Group 2

Written by:

DANICA SHEFFIELD,
SINA ODEJINMI,
STEFAN SPASOVSKI,
AND JENNY NGO

Table of Contents

EXECUTIVE SUMMARY **2**

EVENTBRITE **3**

- Introduction and History 3
- Vision and Objectives 3
- Products/Services 3
- Transactional Databases 4

INITIAL REQUIREMENTS **4**

DATABASE DESIGN **5**

- Conceptual Model 5
- Conceptual Model Description 5
- Logical Model 6
- Logical Model Description 6

DATABASE IMPLEMENTATION **7**

- Physical Model 7

DATABASE DEMONSTRATION **8**

- Final Requirements Matrix 8
- Event Creation 9
- Event Orders 10
- Ticket and Venue Types 11
- Event Browsing 12
- Attendee Tickets 12
- Order Status 13
- Attendee Check-In 13
- Produce Reports 14

CONCLUSION **16**

Executive Summary

Fast changes in digital tools now define how events are run. Instead of printed ads or long lines at desks, people involved turn to connected web tools. Organizers once relied on paper flyers, in-person sales, and handwritten sign-ups. Now, most actions take place within a single shared digital space. Among those options, Eventbrite stands out by offering everything needed under a single platform.

From start to finish, Eventbrite handles every part of an event without extra steps. Payment goes through the system, so nothing falls between the person and the account. People showing up get live updates. While those attending can find events by category and more with just one click. Buying access is done online, and they receive instant digital passes.

What mattered most was building a model around how the system handles key activities - event management and ticket logistics. However, instead of handling every minor detail, attention was focused on core workflows like setting up events, handling user roles, tracking ticket allocations, and managing payments across transactions. With that, the team built a system that closely mirrors industry practices.

What lies behind the structure is a network of tables linking together Event, Organizer, Venue, User, Ticket, Order, and Payment. From one piece to another, information flows about events, available tickets, and money movements.

We are presenting on Eventbrite uses of relational databases to handle large numbers of live event data without losing precision or crashing the system. This work shows the basic structure of data design in Eventbrite's infrastructure of event management.

Eventbrite

Introduction and History

In 2006, Kevin Hartz teamed up with his wife, Julia, along with Renaud Visage, to launch Eventbrite. The purpose was to make planning and attending events less of a hassle. Shows, charity events, and lessons were how people connected at work and outside of it. Yet outdated methods were being used to manage these events. Spreadsheets took over scheduling, paper tickets were the entry method, and payments were split across multiple platforms. Any delay or mistake would cause problems for the organizers.

In San Francisco, Eventbrite was created to help streamline the hassle of event planning. Using Eventbrite enabled the creation of online showrooms where planners could design spaces for their guests. Setting up was easy, with descriptions displayed instantly, buying links available, and money transactions securely managed through the site. Eventbrite has become a top player in event tech worldwide. Millions of organizers rely on it every year for event management.

Vision and Objectives

Eventbrite's vision is to build a community through live experiences and give creators the tools to plan, promote, and create successful events. Eventbrite's objective includes accessibility, by creating a platform that allows users to customize event pages and launch promotional campaigns quickly. Scalability, where the platform supports events ranging from small gatherings to major festivals. Finally, Eventbrite's objectives include security, allowing personal data and financial transactions to be maintained through secure payment processing, and allowing user information to be protected.

Products/Services

Eventbrite provides a multitude of aspects when it comes to event management and ticketing services. It allows organizers to manage registrations and sell tickets by creating multiple ticket types (such as general admissions and VIP), as well as establish capacity limits. The platform also includes secure payment processing, and once a ticket is purchased, attendees receive digital confirmation and scannable tickets, which can be validated at the event venue and help in reducing the need for printed tickets.

Transactional Databases

Transactional databases are fundamental to Eventbrite's daily operations. Because the platform processes high volumes of real-time ticket purchases and user interactions, it relies on structured relational database systems to ensure accuracy, speed, and reliability.

When an organizer creates an event, the system records event details such as date, time, venue, pricing, and capacity. Related tables may include Organizer, Venue, and Category, ensuring that each piece of information is stored efficiently and linked through primary and foreign keys. This relational structure prevents duplication and maintains data consistency.

When an attendee purchases a ticket, multiple database transactions occur almost simultaneously. The system verifies ticket availability, creates an Order record, processes payment information, generates a Ticket record, and updates the event's remaining capacity.

Initial Requirements

For the scope of this project, we worked on Eventbrite's customer-facing application. Our desire for the database is the following:

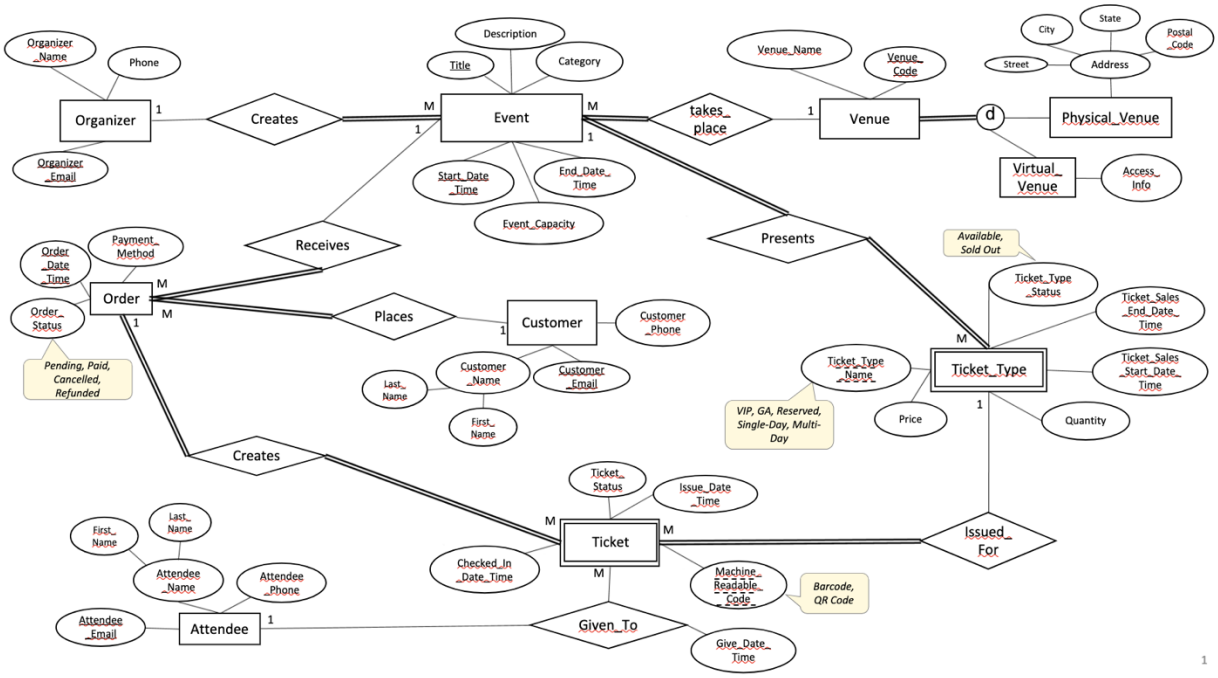
- Users must be able to create and manage an event
- Users must be able to sell and manage tickets to an event
- Users must be able to check in to the event
- Users must be able to browse for events
- Users must be able to purchase tickets for desired events

We started to analyze the possible components of the database, and we were able to identify the following entities:

- Organizer
- Customer
- Event
- Order
- Type of Ticket
- Ticket
- Attendee

Database Design

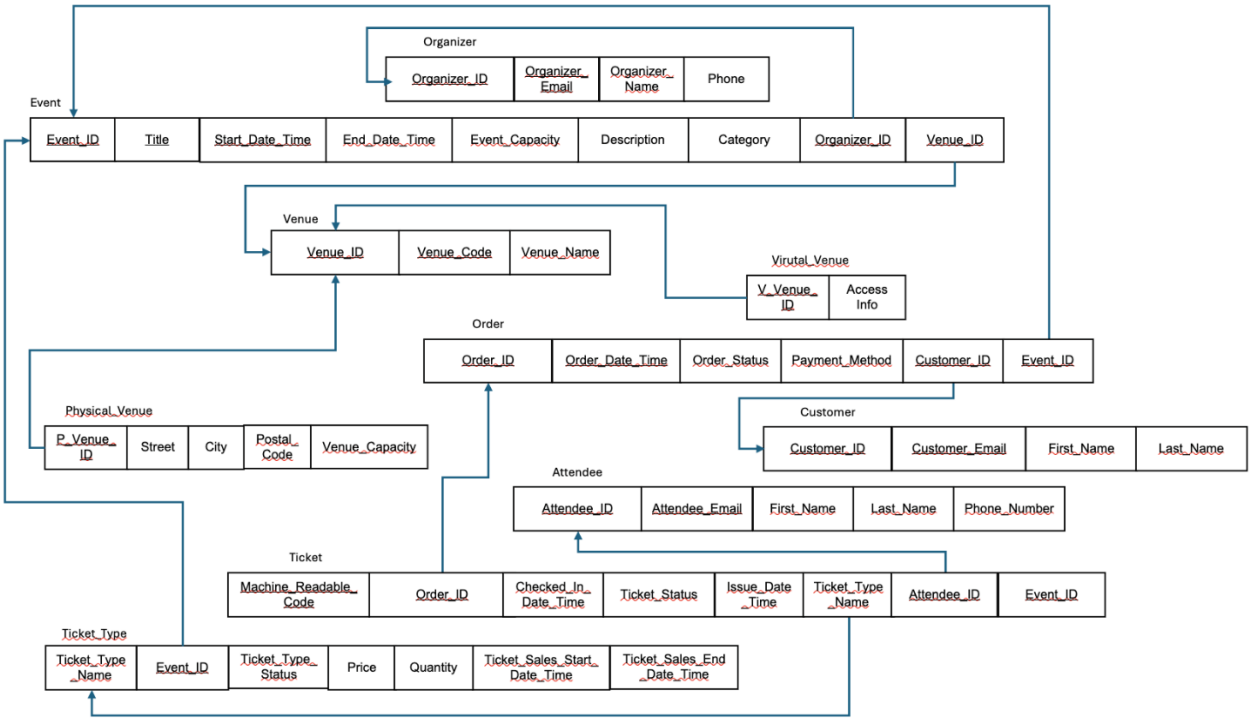
Conceptual Model



Conceptual Model Description

The above diagram is our conceptual model. As Eventbrite is a database revolved around planning and creating events, the main entity is the event. Users can organize an event with a venue, and the database handles ticket sales of various ticket types when an order is made. With that, this diagram depicts the steps involved in creating an event. When the event is being made, an organizer's information will be created. Along with that, the event can be viewed in the database with a name and description, and users can filter it more easily. The event can be a virtual or physical venue, and along with that, different ticket types might be available for the event that can be created. When a ticket is sold, an order is made, and a customer's name will be linked with the order. A specific ticket will be made with that order to make it easier for the event check-in. The attendee was made a different entity than the customer for the possible case when a ticket is bought and given to another individual for use. Overall, the model helps in mapping out the logistics of the database for an organizer who has the capability of creating an event and the different facets in which Eventbrite helps the organizer centralize all the main components needed to make the event successful.

Logical Model

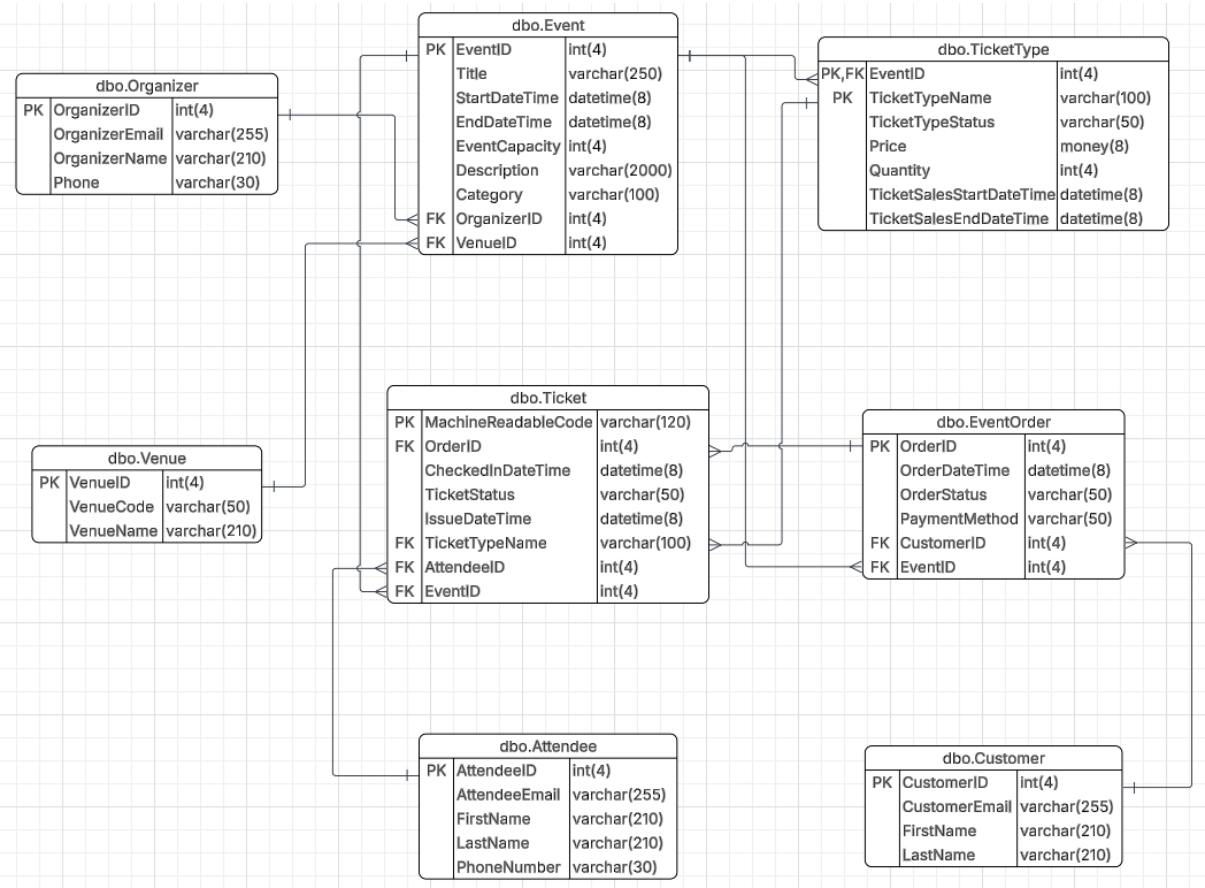


Logical Model Description

The above diagram is our logical model. This model depicts most of the primary keys are specialized IDs for the event, organizer, venue, order, and customer. The reason we choose to show this is that when buying and creating an event, an individualized ID would be linked to the account. When an order is made, a specialized ID will be made to identify the payment and the distinct tickets to be made. Overall, as the event is the main entity, it has the most information stored and is linked to multiple other entities. This logical model helps in organizing and compiling all the various components that Eventbrite uses to help in organizing and managing a multitude of events.

Database Implementation

Physical Model




```sql

CREATE TABLE Organizer (
  OrganizerID INT NOT NULL,
  OrganizerEmail VARCHAR(255) NOT NULL,
  OrganizerName VARCHAR(210) NOT NULL,
  Phone VARCHAR(30),
  CONSTRAINT Organizer_PK PRIMARY KEY (OrganizerID)
);

CREATE TABLE Venue (
  VenueID INT NOT NULL,
  VenueCode VARCHAR(50) NOT NULL,
  VenueName VARCHAR(210) NOT NULL,
  CONSTRAINT Venue_PK PRIMARY KEY (VenueID)
);

CREATE TABLE Event (
  EventID INT NOT NULL,
  Title VARCHAR(250) NOT NULL,
  StartDateTime DATETIME NOT NULL,
  EndDateTime DATETIME NOT NULL,
  EventCapacity INT,
  Description VARCHAR(2000),
  Category VARCHAR(100),
  OrganizerID INT NOT NULL,
  VenueID INT,
  CONSTRAINT Event_PK PRIMARY KEY (EventID),
  CONSTRAINT Event_FK1 FOREIGN KEY (OrganizerID) REFERENCES Organizer(OrganizerID),
  CONSTRAINT Event_FK2 FOREIGN KEY (VenueID) REFERENCES Venue(VenueID)
);

```

```sql
CREATE TABLE Customer (
Cancel query stomerID INT NOT NULL,
  CustomerEmail VARCHAR(255) NOT NULL,
  FirstName VARCHAR(210) NOT NULL,
  LastName VARCHAR(210) NOT NULL,
  CONSTRAINT Customer_PK PRIMARY KEY (CustomerID)
);

CREATE TABLE Attendee (
  AttendeeID INT NOT NULL,
  AttendeeEmail VARCHAR(255) NOT NULL,
  FirstName VARCHAR(210) NOT NULL,
  LastName VARCHAR(210) NOT NULL,
  PhoneNumber VARCHAR(30),
  CONSTRAINT Attendee_PK PRIMARY KEY (AttendeeID)
);
CREATE TABLE EventOrder (
  OrderID INT NOT NULL,
  OrderDateTime DATETIME NOT NULL,
  OrderStatus VARCHAR(50) NOT NULL,
  PaymentMethod VARCHAR(50),
  CustomerID INT NOT NULL,
  EventID INT NOT NULL,
  CONSTRAINT EventOrder_PK PRIMARY KEY (OrderID),
  CONSTRAINT EventOrder_FK1 FOREIGN KEY (CustomerID) REFERENCES Customer(CustomerID),
  CONSTRAINT EventOrder_FK2 FOREIGN KEY (EventID) REFERENCES Event(EventID)
);
CREATE TABLE TicketType (
  EventID INT NOT NULL,
  TicketTypeName VARCHAR(100) NOT NULL,
  TicketTypeStatus VARCHAR(50) NOT NULL,
  Price MONEY NOT NULL,
  Quantity INT NOT NULL,
  TicketSalesStartDateTime DATETIME,
  TicketSalesEndDateTime DATETIME,
  CONSTRAINT TicketType_PK PRIMARY KEY (EventID, TicketTypeName),
  CONSTRAINT TicketType_FK1 FOREIGN KEY (EventID) REFERENCES Event(EventID)
);
CREATE TABLE Ticket (
  MachineReadableCode VARCHAR(120) NOT NULL,
  OrderID INT NOT NULL,
  CheckedInDateTime DATETIME,
  TicketStatus VARCHAR(50) NOT NULL,
  IssueDateTime DATETIME NOT NULL,
  TicketTypeName VARCHAR(100) NOT NULL,
  AttendeeID INT NOT NULL,
  EventID INT NOT NULL,
  CONSTRAINT Ticket_PK PRIMARY KEY (MachineReadableCode),
  CONSTRAINT Ticket_FK1 FOREIGN KEY (OrderID) REFERENCES EventOrder(OrderID),
  CONSTRAINT Ticket_FK2 FOREIGN KEY (AttendeeID) REFERENCES Attendee(AttendeeID),
  CONSTRAINT Ticket_FK3 FOREIGN KEY (EventID) REFERENCES Event(EventID),
  CONSTRAINT Ticket_FK4 FOREIGN KEY (EventID, TicketTypeName)
    REFERENCES TicketType(EventID, TicketTypeName)
);
```

Database Demonstration

Final Requirements Matrix

Category	Description	Status
Event	Organizer can create events	Complete
Event	Customer can browse for event	Complete
Event	Organizer can designate venue/venue type for event	Complete
Event	Organizer can create multiple events at a venue	Complete
Order	Customer can purchase tickets	Complete
Order	Customer can check order status	Complete
Ticket	Organizer can create/issue multiple types of tickets for an event	Complete

Ticket	Ticket will be issued to an attendee and order	Complete
Ticket	Ticket will be recorded and updated for real-time event entry validation	Complete
Ticket	Organizer can track which type of ticket was issued to an attendee	Complete
Payment	Customer can use various payment methods	Complete

Event Creation

Organizers must be able to create events and designate a venue or venue type for the event.

```sql
--Organizer is able to create events

SELECT o.OrganizerName, e.EventID, e.Title, e.StartDateTime, e.EndDateTime
FROM Event101.Event e
JOIN Event101.Organizer o ON e.OrganizerID = o.OrganizerID;

--Organizer can designate venue/venue type for event

SELECT e.Title, v.VenueName, v.VenueCode
FROM Event101.Event e
JOIN Event101.Venue v ON e.VenueID = v.VenueID;
```

Results Messages

Search to filter items...

OrganizerName	EventID	Title	StartDateTime	EndDateTime
NorthStar Events	1	Innovation Summit	2026-04-12T09:00:00.0000000	2026-04-12T17:00:00.0000000
Summit Productions	2	Summer Sounds Live	2026-06-18T15:00:00.0000000	2026-06-18T22:30:00.0000000
NorthStar Events	3	Leadership Workshop	2026-05-08T08:30:00.0000000	2026-05-08T15:30:00.0000000
Community Live	4	Local Makers Market	2026-07-09T10:00:00.0000000	2026-07-09T18:00:00.0000000
Community Live	5	Design & Creativity Expo	2026-09-14T10:00:00.0000000	2026-09-14T16:00:00.0000000
Summit Productions	6	Data Analytics Bootcamp	2026-10-03T09:00:00.0000000	2026-10-03T16:00:00.0000000
Utah Creatives	10	Art in the Park	2026-10-01T10:00:00.0000000	2026-10-01T17:00:00.0000000

```sql
--Organizer can designate venue/venue type for event

SELECT e.Title, v.VenueName, v.VenueCode
FROM Event101.Event e
JOIN Event101.Venue v ON e.VenueID = v.VenueID;

--An organizer can create multiple events at a venue

SELECT v.VenueName, COUNT(e.EventID) AS NumberOfEvents
FROM Event101.Venue v
LEFT JOIN Event101.Event e ON v.VenueID = e.VenueID
GROUP BY v.VenueName
HAVING COUNT(e.EventID) > 1;
```

Results Messages

Search to filter items...

Title	VenueName	VenueCode
Innovation Summit	Central Convention Center	CNV-01
Summer Sounds Live	Lakeside Amphitheater	OUT-04
Leadership Workshop	Grand Theater	THR-02
Local Makers Market	Riverside Hall	HALL-03
Design & Creativity Expo	Central Convention Center	CNV-01
Data Analytics Bootcamp	Grand Theater	THR-02
Art in the Park	Lakeside Amphitheater	OUT-04

A new organizer is made and is registered in the database with an id, email, phone, and name. The organizer will link their id with an event that they create, allowing the organizer to determine the title, start date and time, end date and time, description, category, venue, and event capacity. The sample demonstrates an organizer's ability to create an event and how it is linked to the organizer, as well as how a venue is designated to an event.

Event Orders

The customer should be able to purchase tickets for events and use various payment methods.

```sql
--Customer is able to purchase tickets

SELECT c.FirstName + ' ' + c.LastName AS CustomerName, eo.OrderID, eo.OrderStatus, e.Title AS EventTitle
FROM Event101.EventOrder eo
JOIN Event101.Customer c ON eo.CustomerID = c.CustomerID
JOIN Event101.Event e ON eo.EventID = e.EventID
WHERE eo.OrderStatus = 'Completed';

--Customer is able to check order status

SELECT OrderID, OrderStatus, OrderDateTime
FROM Event101.EventOrder
WHERE CustomerID = 1;
```

Results Messages

Search to filter items...

CustomerName	OrderID	OrderStatus	EventTitle
Jordan Adams	1001	Completed	Innovation Summit
Morgan Turner	1002	Completed	Innovation Summit
Casey Hall	1003	Completed	Summer Sounds Live
Taylor Wright	1004	Completed	Leadership Workshop
Alex Reed	1005	Completed	Local Makers Market
Kris Evans	1006	Completed	Design & Creativity Expo
Jordan Adams	1007	Completed	Summer Sounds Live
Morgan Turner	1008	Completed	Data Analytics Bootcamp

```sql
GROUP BY Event.EventID, Event.Title, Event.Category, Event.StartDateTime, Event.EndDateTime, Venue.VenueName
ORDER BY Event.StartDateTime ASC;

-- Customer can use various payment methods

SELECT EventOrder.OrderID,
       EventOrder.OrderDateTime,
       EventOrder.OrderStatus,
       EventOrder.PaymentMethod,
       Customer.CustomerEmail,
       Event.Title
FROM EventOrder
JOIN Customer
  ON EventOrder.CustomerID = Customer.CustomerID
JOIN Event
  ON EventOrder.EventID = Event.EventID
WHERE EventOrder.PaymentMethod IS NOT NULL
ORDER BY EventOrder.OrderDateTime ASC;

-- Organizer can create multiple events at a venue
```

ults Messages

Search to filter items...

derID	OrderDateTime	OrderStatus	PaymentMethod	CustomerEmail	Title
01	2026-03-01T09:10:00.0000000	Completed	Credit Card	j.adams@email.com	Innovation Summit
02	2026-03-02T11:25:00.0000000	Completed	PayPal	m.turner@email.com	Innovation Summit
03	2026-03-03T14:40:00.0000000	Completed	Credit Card	c.hall@email.com	Summer Sounds Live
04	2026-03-04T16:05:00.0000000	Pending	Debit Card	t.wright@email.com	Leadership Workshop
05	2026-03-05T10:20:00.0000000	Completed	Credit Card	a.reed@email.com	Local Makers Market
06	2026-03-06T13:55:00.0000000	Completed	PayPal	k.evans@email.com	Design & Creativity Expo
07	2026-03-07T09:45:00.0000000	Completed	Credit Card	j.adams@email.com	Summer Sounds Live

The first screenshot details a customer who has made a purchase. The name is linked with an order ID for a specific event, and the order status is marked as purchase. Demonstrating that customers can order and purchase tickets. The second screenshot demonstrates that customers are able to use either a credit card, PayPal, or even debit cards to purchase on Eventbrite. This depicts that customers can use a variety of payment methods when purchasing a ticket for an event.

Ticket and Venue Types

The organizer should be able to issue multiple ticket types for an event and create multiple events at a venue.

```sql
--Organizer can create issue multiple types of tickets for an event

SELECT e.Title, tt.TicketTypeName, tt.Price, tt.TicketTypeStatus
FROM Event101.TicketType tt
JOIN Event101.Event e ON tt.EventID = e.EventID
ORDER BY e.EventID;

--Ticket will be issued to an attendee and order
SELECT t.MachineReadableCode, a.FirstName + ' ' + a.LastName AS AttendeeName,
eo.OrderID, eo.OrderStatus, e.Title AS EventTitle
FROM Event101.Ticket t
```

Results Messages

Search to filter items...

Title	TicketTypeName	Price	TicketTypeStatus
Innovation Summit	General Admission	65.0000	Active
Innovation Summit	VIP Access	150.0000	Active

```sql
--An organizer can create multiple events at a venue

SELECT v.VenueName, COUNT(e.EventID) AS NumberOfEvents
FROM Event101.Venue v
LEFT JOIN Event101.Event e ON v.VenueID = e.VenueID
GROUP BY v.VenueName
HAVING COUNT(e.EventID) > 1;

--Customer is able to purchase tickets

SELECT c.FirstName + ' ' + c.LastName AS CustomerName, eo.OrderID, eo.OrderStatus, e.Title AS EventTitle
FROM Event101.EventOrder eo
JOIN Event101.Customer c ON eo.CustomerID = c.CustomerID
```

Results Messages

Search to filter items...

VenueName	NumberOfEvents
Central Convention Center	2
Grand Theater	2
Lakeside Amphitheater	2

The first portion of the screenshot demonstrates that for the innovation summit event, there are two different types of tickets being issued for the event. VIP Access, as well as General Admission, is available for customers to purchase. While the second screenshot

demonstrates that multiple events can be created at a venue. With this, these accommodate different venues and tickets that the organizer might be creating for an event.

Event Browsing

```sql
-- Customer is able to browswer for event (shows upcoming events with venue, category, dates, and ticket price)

SELECT Event.EventID,
       Event.Title,
       Event.Category,
       Event.StartDateTime,
       Event.EndDateTime,
       Venue.VenueName,
       MIN(TicketType.Price) AS LowestTicketPrice,
       MAX(TicketType.Price) AS HighestTicketPrice
FROM Event
LEFT JOIN Venue
  ON Event.VenueID = Venue.VenueID
LEFT JOIN TicketType
  ON Event.EventID = TicketType.EventID
WHERE Event.StartDateTime >= GETDATE()
GROUP BY Event.EventID, Event.Title, Event.Category, Event.StartDateTime, Event.EndDateTime, Venue.VenueName
ORDER BY Event.StartDateTime ASC;

-- Customer can use various payment methods
```

Results Messages

Search to filter items...

EventID	Title	Category	StartDateTime	EndDateTime	VenueName	LowestTicketPrice	HighestTicketPrice
1	Innovation Summit	Technology	2026-04-12T09:00:00.0000000	2026-04-12T17:00:00.0000000	Central Convention Center	65.0000	150.0000
3	Leadership Workshop	Business	2026-05-08T08:30:00.0000000	2026-05-08T15:30:00.0000000	Grand Theater	45.0000	45.0000
2	Summer Sounds Live	Music	2026-06-18T15:00:00.0000000	2026-06-18T22:30:00.0000000	Lakeside Amphitheater	85.0000	160.0000
4	Local Makers Market	Community	2026-07-09T10:00:00.0000000	2026-07-09T18:00:00.0000000	Riverside Hall	20.0000	20.0000
5	Design & Creativity Expo	Art	2026-09-14T10:00:00.0000000	2026-09-14T16:00:00.0000000	Central Convention Center	30.0000	30.0000
99	Demo Event	Demo	2026-10-01T10:00:00.0000000	2026-10-01T12:00:00.0000000	Demo Hall	10.0000	10.0000

When the events are first created, a title, category, start and end date, and more are entered. With that information, users are able to group the events into various categories for easier browsing. Making it easier for users to find certain events or similar events that they might be interested in.

Attendee Tickets

All tickets issued should be tied to an order, assigned to an attendee, and belonging to a specific ticket type.

```sql
--Ticket will be issued to an attendee and order
SELECT t.MachineReadableCode, a.FirstName + ' ' + a.LastName AS AttendeeName,
eo.OrderID, eo.OrderStatus, e.Title AS EventTitle
FROM Event101.Ticket t
JOIN Event101.Attendee a ON t.AttendeeID = a.AttendeeID
JOIN Event101.EventOrder eo ON t.OrderID = eo.OrderID
JOIN Event101.Event e ON t.EventID = e.EventID;

--Ticket will be recorded and updated for real-time event entry validation
SELECT MachineReadableCode, TicketStatus, CheckedInDateTime
FROM Event101.Ticket
WHERE CheckedInDateTime IS NOT NULL;
```

Results Messages

Search to filter items...

MachineReadableCode	AttendeeName	OrderID	OrderStatus	EventTitle
T-1001-A	Jordan Adams	1001	Completed	Innovation Summit
T-1001-B	Emily Brooks	1001	Completed	Innovation Summit

Each ticket should be tied to an order, assigned to an attendee, and belonging to a specific ticket type. The order status is marked as complete when the transaction is finished, and a

ticket with a machine-readable code is made and assigned to an attendee. This makes it possible for the attendees to get guaranteed access to the event and makes sure no duplicate tickets are created.

Order Status

The database should allow for order status changes to occur, such as from Pending to Complete.

```sql
--Customer is able to check order status

SELECT OrderID, OrderStatus, OrderDateTime
FROM Event101.EventOrder
WHERE CustomerID = 1;

--Organizer can create issue multiple types of tickets for an event

SELECT e.Title, tt.TicketTypeName, tt.Price, tt.TicketTypeStatus
FROM Event101.TicketType tt
JOIN Event101.Event e ON tt.EventID = e.EventID
ORDER BY e.EventID;

--Ticket will be issued to an attendee and order
```

Results Messages

Search to filter items...

OrderID	OrderStatus	OrderDateTime
1001	Completed	2026-03-01T09:10:00.0000000
1007	Completed	2026-03-07T09:45:00.0000000

To hold events, sometimes in the process of purchasing a ticket, the order should be in a pending status. Once an order is completed, a ticket will finally be issued to an attendee, and they will be updated in the system. With that, organizers can check the order status, as well as allow customers to later check to make sure that the order status is completed. Ensuring that once an order is completed and an update is made in the system, the following steps can be taken to make sure the customer is able to attend the event.

Attendee Check-In

The system should record check-in times for real-time event entry validation.

```sql
--Ticket will be recorded and updated for real-time event entry validation
SELECT MachineReadableCode, TicketStatus, CheckedInDateTime
FROM Event101.Ticket
WHERE CheckedInDateTime IS NOT NULL;

--Organizer is able to track which type of ticket was issued to an attendee
SELECT a.FirstName + ' ' + a.LastName AS AttendeeName, t.TicketTypeName, e.Title AS EventTitle
FROM Event101.Ticket t
JOIN Event101.Attendee a ON t.AttendeeID = a.AttendeeID
JOIN Event101.Event e ON t.EventID = e.EventID;
```

Results Messages

Search to filter items...

MachineReadableCode	TicketStatus	CheckedInDateTime
T-3001-A	Checked In	2026-03-05T06:07:46.5570000

To make sure that a ticket isn't used multiple times to check into the event. The system records when a code is used and marks the time and date at which it was scanned. When that happens, the ticket is marked as check-in and cannot be reused to make sure that it isn't used again by another individual to check into the event.

Produce Reports

The organizer should be able to create reports for more in-depth details about the event.

```sql
--Organizer is able to track which type of ticket was issued to an attendee
SELECT a.FirstName + ' ' + a.LastName AS AttendeeName, t.TicketTypeName, e.Title AS EventTitle
FROM Event101.Ticket t
JOIN Event101.Attendee a ON t.AttendeeID = a.AttendeeID
JOIN Event101.Event e ON t.EventID = e.EventID;
```

Results Messages

Search to filter items...

AttendeeName	TicketTypeName	EventTitle
Jordan Adams	General Admission	Innovation Summit
Emily Brooks	VIP Access	Innovation Summit

```sql
--Organizer is able to track how many tickets were sold
SELECT EventID, COUNT(*) AS TicketsSold
FROM Event101.Ticket
GROUP BY EventID;
```

Results Messages

Search to filter items...

EventID	TicketsSold
1	3

```sql
--Count tickets sold per event
SELECT EventID, COUNT(*) AS TicketsSold
FROM Event101.Ticket
GROUP BY EventID;
```

SELECT* FROM EVENT101.Organizer

EventID	TicketsSold
1	3
2	4
3	1

```
--Find all attendees who bought VIP Access for Innovation Summit

SELECT a.AttendeeID, a.FirstName, a.LastName
FROM Event101.Ticket t
JOIN Event101.Attendee a ON t.AttendeeID = a.AttendeeID
WHERE t.EventID = 1
  AND t.TicketTypeName = 'VIP Access';
```

AttendeeID	FirstName	LastName
2	Emily	Brooks

As an organizer, a concise report is needed to decide on the success of the event. The above screenshots depict the various reports that can be compiled about the events. The organizer is able to see how many tickets were sold for a specific event or multiple events. Along with that, an organizer can compile a list of attendees who bought VIP Access to an event, for when they need to follow up with further information about seats and perks that come with the VIP ticket. They can also bring up all the customer information and find out which attendees bought which type of ticket and determine other factors such as capacity and price for the next event. Overall, these reports are a good way for organizers to see data that was gathered when a customer buys a ticket. The system considers that and allows the organizer to compile the data that was collected for the event.

Conclusion

Eventbrite is an organization that is working to connect people to live events and experiences. Believing that live events can bring like-minded individuals together and create a community in which they interact with one another. From this project, we can conclude that there are many working parts for this database to ensure that an event runs smoothly and safely for its users. Along with that, the database also allows for so many different events that the various components, like ticket types and venues, need to be adjusted to accommodate these events. As the world of technology continues to develop, live events are a way to connect to one another, and Eventbrite will likely continue this for years to come.